



Universidad de Granada

decsai.ugr.es

Inteligencia Artificial

Seminario 3

Búsqueda con Adversario (Juegos)



DECSAI

**Departamento de Ciencias de la
Computación e Inteligencia Artificial**

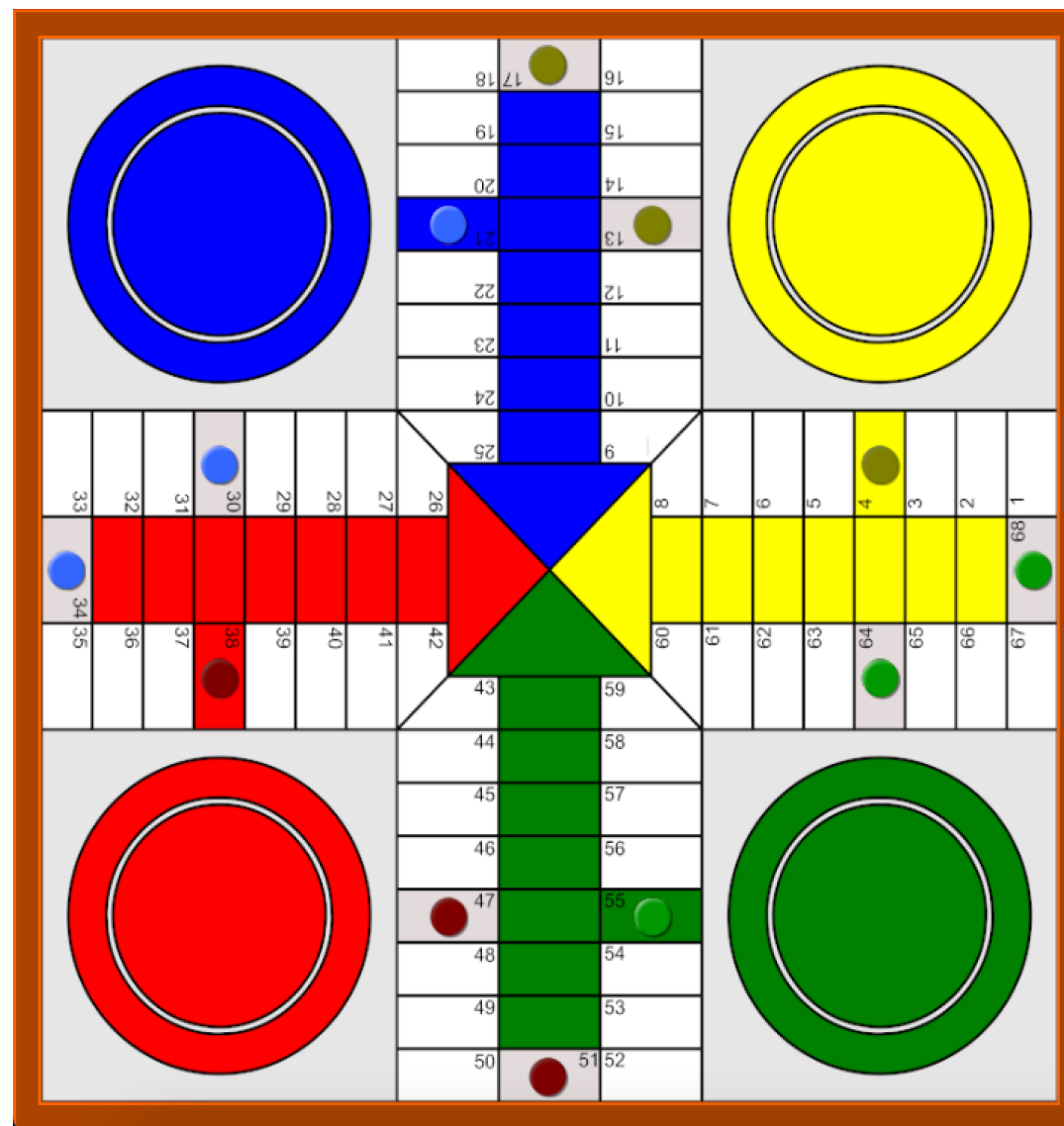
- 1. Introducción**
- 2. Presentación del Juego**
- 3. Presentación del Simulador**
- 4. Diseño y desarrollo**
- 5. Método de evaluación de la práctica**



- 1. Introducción**
2. Presentación del Juego
3. Presentación del Simulador
4. Diseño y desarrollo
5. Método de evaluación de la práctica



Diseñar e implementar técnicas de búsqueda con adversario en un entorno de juegos.



- El parchís es un juego de mesa derivado del pachisi.
- En su versión original es un juego de 2 a 4 jugadores.
- Hay 4 casas de colores: amarillo, rojo, verde y azul.
- Cada jugador tiene 4 fichas del color de su casa.
- Dispone de un tablero con un circuito de 100 casillas.
- Objetivo del juego: Ser el primero en llevar las 4 fichas desde la casa hasta la meta recorriendo todo el circuito, siendo posible “comer” las fichas de los oponentes.

- Tradicionalmente, en el juego del parchís:
 - Cada jugador dispone de un color de juego.
 - Se dispone de un dado que permite mover las fichas de juego.

- En esta práctica:
 - Cada jugador dispone de dos colores alternos, pudiendo usar cualquiera de ellos en su turno.
 - Se presenta un Parchís determinista de forma que los dados se van eligiendo entre un conjunto de dados disponibles.
 - Cada color tiene solamente **3 fichas**.
 - Hay alguna que otra cosa especial...

Se va a disponer de un dado especial, que dependiendo de la energía que tengamos en el momento, hará una cosa u otra (no sabemos si buena o mala)...



- Para realizar la práctica, hay que conocer las técnicas de búsqueda con adversario explicadas en teoría.

El objetivo de la práctica será:

*Diseñar e implementar **el algoritmo MINIMAX o el algoritmo de PODA ALFA-BETA** para dotar de un comportamiento inteligente deliberativo a un jugador artificial para este juego.*

Requisitos:

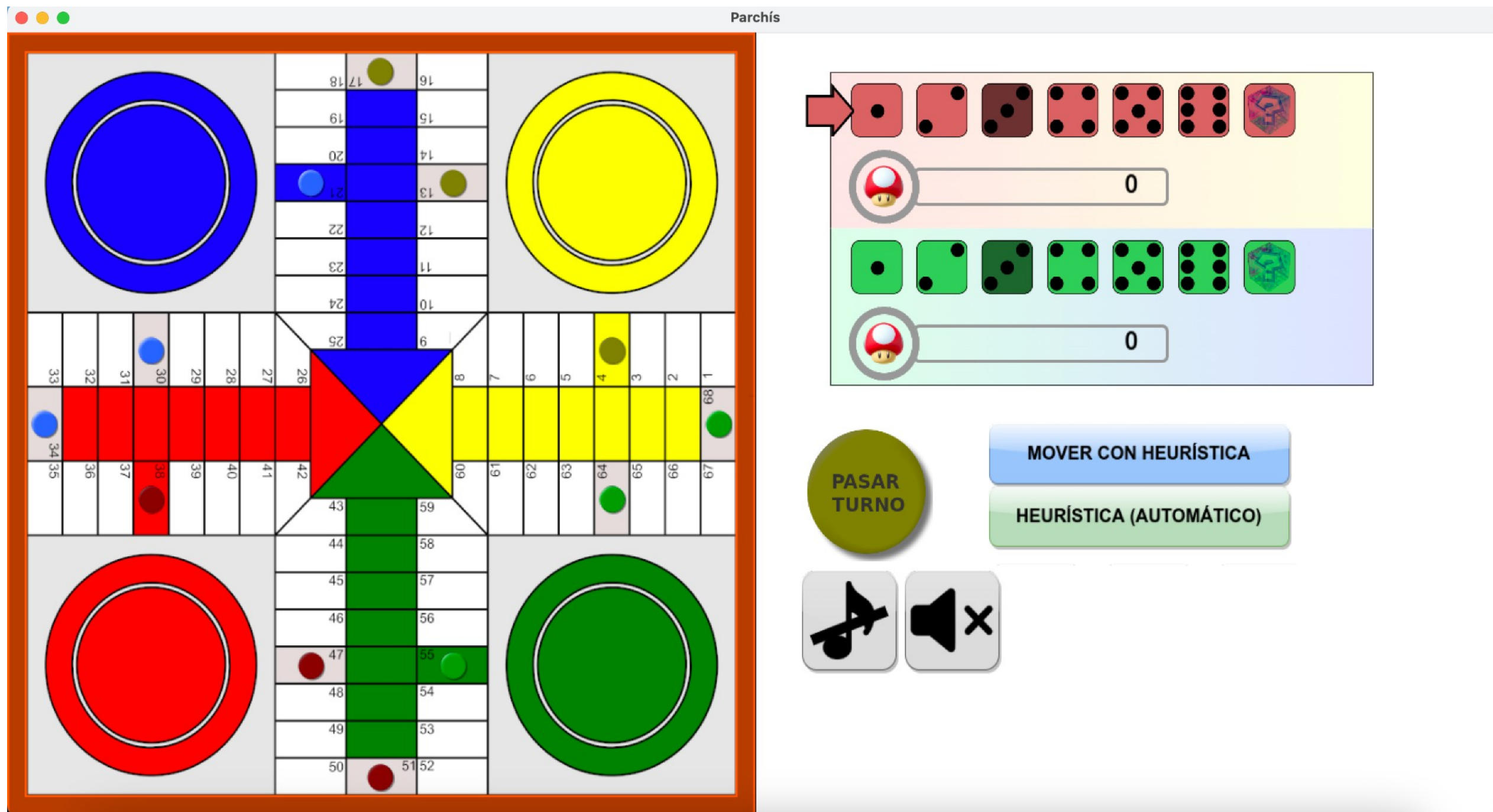
- Conocimientos básicos del lenguaje C/C++
- La librería SFML
 - Ubuntu: `sudo apt install libsFML-dev`
 - MacOS: `brew install sfml`
- El simulador del juego que está disponible en GitHub:
<https://github.com/ugr-ccia-IA/practica3>

El simulador ha sido desarrollado por los profesores de la asignatura.

1. Introducción
- 2. Presentación del Juego**
3. Presentación del Simulador
4. Diseño y desarrollo
5. Método de evaluación de la práctica



EL JUEGO



Jugadores:

- Hay 2 jugadores con dos colores alternos.
- Los turnos se gestionan por jugadores, y no por colores.
- Una ficha de un color puede “comerse” fichas de cualquier otro color, aunque sean del mismo jugador.

Dados:

- Se cambia el comportamiento aleatorio de tirar un dado por la elección del dado entre un conjunto de dados disponibles.
- Inicialmente cada jugador tiene 6 dados que se corresponden con los números 1, 2, 4, 5 y 6, y con un **dado especial**.
- Al usar un dado, ese valor se gasta. Cuando se gastan los 6 dados, se regeneran.
- El funcionamiento del dado especial dependerá del valor de la **energía** en ese momento concreto.

Dados:



©

Normas del juego:

1. Juegan dos jugadores con dos colores cada uno:
Jugador 1 (amarillo y rojo) vs Jugador 2 (azul y verde)
2. En cada turno, el jugador elige un dado y elige la ficha que quiere mover de entre sus 6 fichas. Ese dado se gasta y no vuelve a aparecer hasta que se usen todos.
3. Hay que usar un 5 para sacar una ficha de casa.
4. Si se saca un 6 se vuelve a tirar.
6. Cuando se saca un 6 y todas las fichas de ese color están fuera de casa, se avanza 7 casillas en lugar de 6.

Normas del juego:

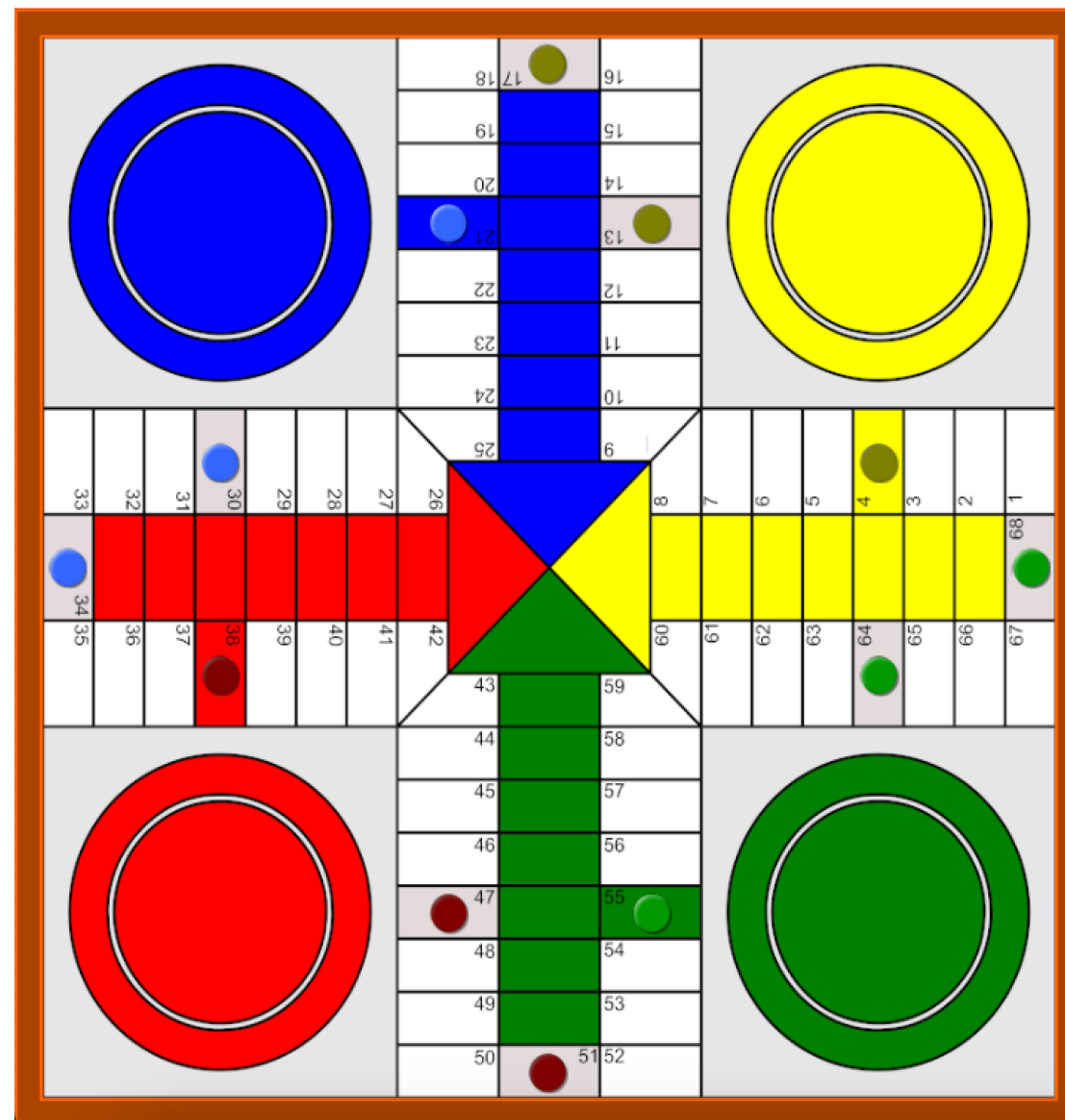
7. No puede haber más de dos fichas en una casilla, salvo en las de casa o meta.
8. Dos fichas del mismo color en una casilla forman una barrera. Una ficha de otro color no puede pasar la barrera. Una ficha del mismo color sí puede saltarla.
9. Si se saca un 6 y hay alguna barrera del color que toca, es obligatorio romper la barrera.
10. Cuando una ficha llega a una casilla no segura donde hay una ficha de otro color, se come esa ficha (vuelve a su casa). El jugador que se come la ficha mueve 20 casillas adelante la ficha que desee de su color.
11. Dos colores de un mismo jugador pueden comerse.

Normas del juego:

12. En las casillas seguras pueden situarse dos fichas de distinto color. No actúan como barrera, por lo que cualquier otra ficha puede saltarse esa casilla.
13. Para entrar a la meta hay que sacar el número exacto de casillas que falten para llegar. Si se saca más, la ficha rebota y cuenta hacia atrás el exceso de casillas. Se establece un máximo de 30 rebotes.
14. Cuando una ficha llega a la meta, se mueve 10 casillas adelante la ficha que desee de su jugador.
15. Si se elige un dado y no se puede mover ninguna ficha, se puede gastar ese dado y pasar el turno sin que el jugador haga movimientos.

Normas del juego:

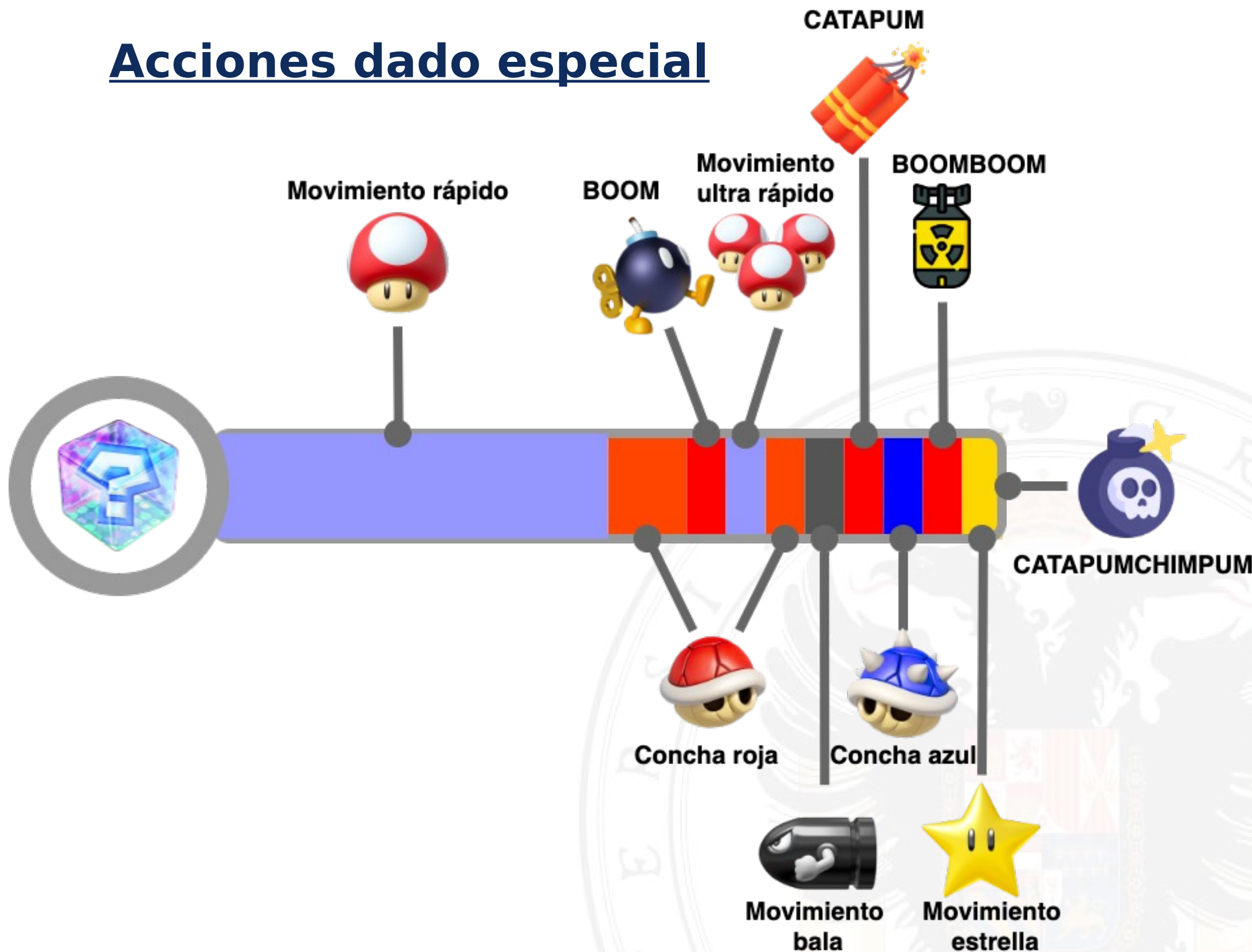
16. Inicialmente las fichas no aparecen todas en casa sino organizadas del siguiente modo:



Barra de energía

- **Se consume:** cuando se hace uso del dado especial, quedando la barra de energía a 0 al finalizar ese turno.
- **Se recarga:** cuando se realiza cualquier otro movimiento, la energía se incrementa en tantos puntos como casillas se haya avanzado. Además:
 - +5 puntos por casilla segura.
 - +10 puntos por formar una nueva barrera.
 - +15 puntos cuando la ficha que se mueve se come a otra ficha.
- La energía máxima se establece a **100** para cada jugador.

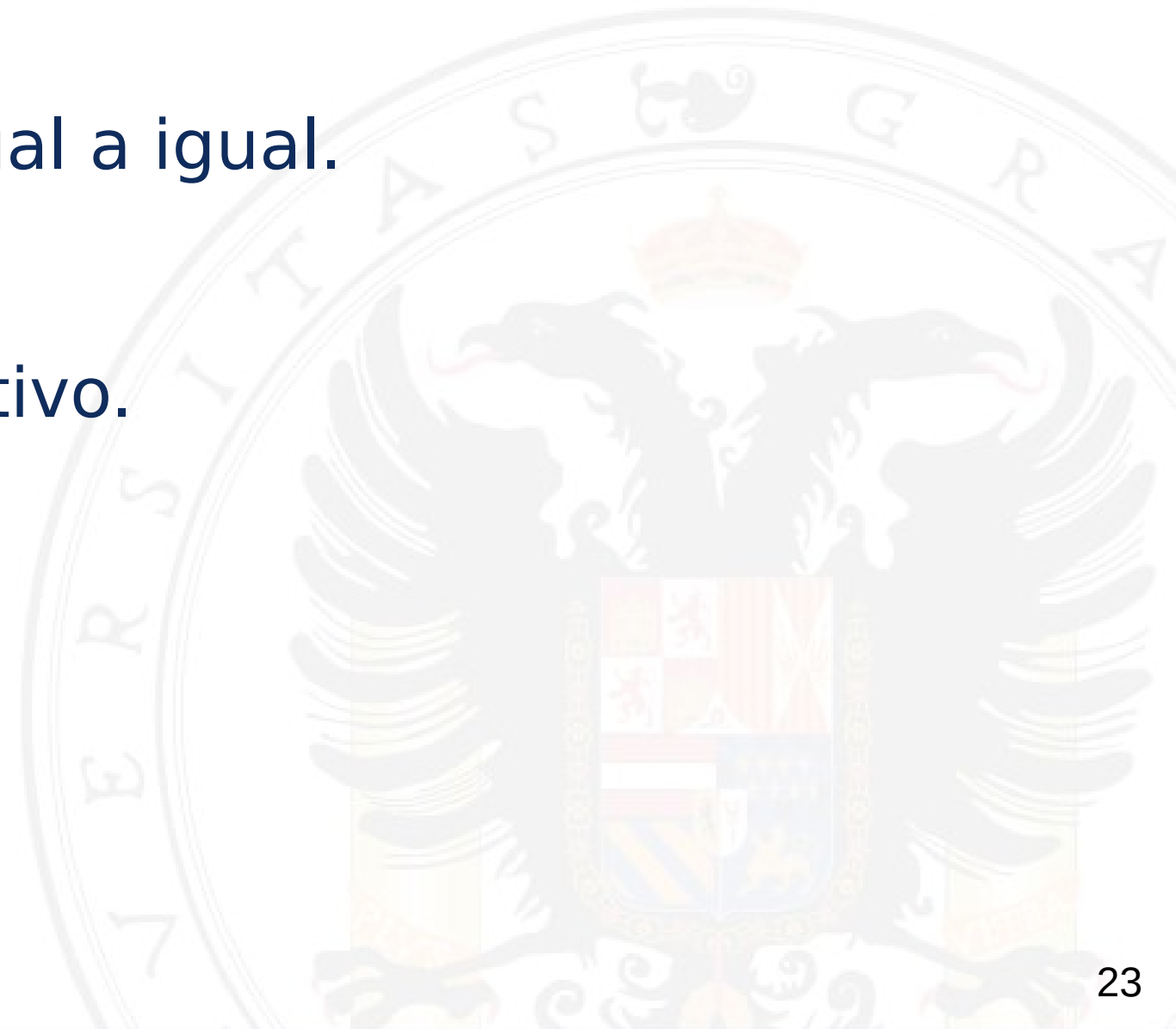
Acciones dado especial



El objetivo de la práctica es implementar el algoritmo **MINIMAX (con profundidad máxima de 4)** o el algoritmo de **PODA ALFA-BETA (con profundidad máxima 6)** de manera que un jugador pueda determinar el movimiento más prometedor para ganar el juego, explorando el árbol de juego desde el estado actual hasta una profundidad máxima de 4 o 6 según el algoritmo.

Es muy importante la **definición de la heurística.**

- **Ninja 1:** asequible?
- **Ninja 2:** una batalla de igual a igual.
- **Ninja 3:** el combate definitivo.



1. Introducción
2. Presentación del Juego
- 3. Presentación del Simulador**
4. Diseño y desarrollo
5. Método de evaluación de la práctica



- Se proporciona el software para el sistema operativo Linux en el repositorio de GitHub
<https://github.com/ugr-ccia-IA/practica3>
 - Hay dos opciones de ejecución:
 - Simulador con interfaz gráfica
 - Simulador en modo *batch* sin interfaz.
 - La explicación sobre el uso del simulador se presenta en la sección 4 del **guión de prácticas** asociado a esta presentación.
- Ver simulador –

1. Introducción
2. Presentación del Juego
3. Presentación del Simulador
- 4. Diseño y desarrollo**
5. Método de evaluación de la práctica



- Sólo hay 2 ficheros relevantes para la práctica:
 - AIPlayer.cpp
 - AIPlayer.h
- Se encuentran en las carpetas src e include.
- El resto de archivos que se adjuntan con la práctica son necesarios y no se pueden modificar.

– Ver tutorial –

1. Introducción
2. Presentación del Juego
3. Presentación del Simulador
4. Diseño y desarrollo
- 5. Método de evaluación de la práctica**



- Se pide desarrollar un programa (modificando el código de los ficheros del simulador 'AIPlayer.cpp' y 'AIPlayer.hpp') que implemente el algoritmo MINIMAX (con profundidad máxima de 4) o la PODA ALFA-BETA (con profundidad máxima de 6).
- Estos dos ficheros deberán entregarse mediante la plataforma web de la asignatura, en un fichero ZIP (que no contenga carpetas ni sub-carpetas).
- El archivo ZIP deberá contener:
 - El código fuente de los dos ficheros previamente descritos.
 - Una memoria de prácticas en formato PDF que describa el comportamiento implementado con un máximo de 5 páginas.

La práctica se califica de **0 a 10 pts.** como la suma de los siguientes criterios:

- La memoria de práctica se evalúa de **0 a 3 ptos.**
- La elección del método de búsqueda se evalúa de **0 a 1 ptos.:**
 - 0 pto.: si usa MINIMAX
 - 1 pto.: si usa una Poda ALFA-BETA
- La eficacia del algoritmo se evalúa de 0 a 6 ptos.:
 - 6 partidas contra 3 ninjas siendo primer y segundo jugador.
 - Cada victoria es 1 pto.

Es requisito necesario entregar tanto el software como la memoria de prácticas. Si alguna de ellas falta la entrega se entenderá por no entrega.

Fecha límite entrega

9 de Junio hasta las 23:00 horas

Esta práctica es INDIVIDUAL

En el caso de detectar prácticas copiadas, los involucrados (tanto el que se copió como el que se ha dejado copiar) tendrán suspensa la asignatura.